Set 2 - Endsem Lab Exam - Set A

Instructions:

1. Attempt **any one** of the questions given below.
2. Use only the following editors: `nano`, `vim`, `gedit`, or `nedit`.
3. You can write your code in C/C++.
4. File naming in this format - **SetNo_QNo_RollNo.c/cpp**, e.g. 2_1_2025123.c or 2_1_2025123.cpp
5. **Google Form Link For Submission: https://forms.gle/sV8VeqWEg5tfphAp9**

Question 1: Implement Priority Queue using Heap

Write a C program to implement a **Priority Queue** using a **Max Heap** with an array-based implementation. The heap must satisfy the max-heap property, where every parent node is greater than or equal to its children. All operations must be performed using the array representation of a binary heap. The priority queue should support `insert(x)` for inserting an element into the priority queue, `heapMaximum()` for returning the highest priority element without removing it, and `heapExtractMax()` for removing and returning the element with the highest priority. Use the helper function `heapIncreaseKey(i, key)` to increase the value of an element at index `i` to a new key and restore the max-heap property by moving the element upward during insertion.

Base Code

```c
#include <stdio.h>
#include <stdlib.h>
#define NEG_INF -1000000000
struct Heap {
   int* array;
   int capacity;
   int size;
};
struct Heap* createHeap(int capacity) {
   struct Heap* heap = (struct Heap*)malloc(sizeof(struct Heap));
   heap->array = (int*)malloc(capacity * sizeof(int));
   heap->capacity = capacity;
   heap->size = 0;
   return heap;}

void swap(int* a, int* b) {
   int temp = *a;
   *a = *b;
   *b = temp;}

int heapMaximum(struct Heap* heap) {
   /* Write your code here */
   return -1;}

int heapExtractMax(struct Heap* heap) {
   /* Write your code here */
   return -1;}

void heapIncreaseKey(struct Heap* heap, int index, int key) { /* Write your code here */}

void insert(struct Heap* heap, int value) {/* Write your code here */}

void maxHeapify(struct Heap* heap, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < heap->size && heap->array[left] > heap->array[largest])
      largest = left;

   if (right < heap->size && heap->array[right] > heap->array[largest])
      largest = right;

   if (largest != i) {
      swap(&heap->array[i], &heap->array[largest]);
      maxHeapify(heap, largest);
   }}

void printHeap(struct Heap* heap) {
   for (int i = 0; i < heap->size; i++)
      printf("%d ", heap->array[i]);
   printf("\n");}

int main() {
   struct Heap* heap = createHeap(100);
   insert(heap, 40);
   insert(heap, 20);
   insert(heap, 50);
   insert(heap, 10);
   insert(heap, 30);

   printf("Heap: ");
   printHeap(heap);

   printf("Max: %d\n", heapMaximum(heap));

   printf("Extract: %d\n", heapExtractMax(heap));
   printHeap(heap);

   printf("Extract: %d\n", heapExtractMax(heap));
```

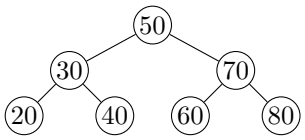
```
71    printHeap(heap);
72    return 0;}
```

Question 2: Delete Node from Binary Search Tree (BST)

Write a C program to perform **deletion of a node in a Binary Search Tree (BST)**. The deletion must be implemented using a helper function `transplant(T, u, v)`, which replaces the subtree rooted at node `u` with the subtree rooted at node `v`. Your program must correctly handle all the cases, which include **leaf node**, **node with one child**, and **node with two children**.

Use the following operations to test your program:

1. Insert: 50, 30, 70, 20, 40, 60, 80
2. Delete: 20 (leaf node)
3. Delete: 30 (node with one child)
4. Delete: 50 (node with two children)



Base Code

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node{
   int key;
   struct Node *left,*right,*p;
} Node;
typedef struct Tree{
   Node *root;
} Tree;

Node* newNode(int key){
   Node* n=(Node*)malloc(sizeof(Node));
   n->key=key; n->left=NULL; n->right=NULL; n->p=NULL;
   return n;}

void insertNode(Tree* T,int key){
   Node* z=newNode(key);
   Node* x=T->root; Node* y=NULL;
   while(x!=NULL){
      y=x;
      if(z->key<x->key) x=x->left;
      else x=x->right;
   }
   z->p=y;
   if(y==NULL) T->root=z;
   else if(z->key<y->key) y->left=z;
   else y->right=z;}

Node* treeSearch(Node* x,int k){
   if(x==NULL || k==x->key) return x;
   if(k<x->key) return treeSearch(x->left,k);
   else return treeSearch(x->right,k);}

Node* treeMin(Node* x){
   while(x->left!=NULL) x=x->left;
   return x;}

void transplant(Tree* T,Node* u,Node* v){/* Write your code here */}

void treeDelete(Tree* T,Node* z){/* Write your code here */}

void inorder(Node* x){
   if(x==NULL) return;
   inorder(x->left);
   printf("%d -> ",x->key);
   inorder(x->right);}

int main(){
   int i;
   int arr[]={50,30,70,20,40,60,80};
   int deleteKeys[]={20,30,50};

   Tree T; T.root=NULL;

   for(i=0;i<7;i++) insertNode(&T,arr[i]);

   printf("Initial inorder traversal: ");
   inorder(T.root); printf("NULL\n");

   for(i=0;i<3;i++){
      Node* z=treeSearch(T.root,deleteKeys[i]);
      if(z==NULL){
         printf("\nKey %d not found in the tree.\n",deleteKeys[i]);
      } else {
         printf("\nDeleting %d:\n",deleteKeys[i]);
         treeDelete(&T,z);
         printf("Inorder traversal after deletion: ");
         inorder(T.root); printf("NULL\n");
      }
   }
   return 0;}
```

Solution 1

```c
#include <stdio.h>
#include <stdlib.h>
#define NEG_INF -1000000000

struct Heap {
   int* array;
   int capacity;
   int size;
};

struct Heap* createHeap(int capacity) {
   struct Heap* heap = (struct Heap*)malloc(sizeof(struct Heap));
   heap->array = (int*)malloc(capacity * sizeof(int));
   heap->capacity = capacity;
   heap->size = 0;
   return heap;
}

void swap(int* a, int* b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

int heapMaximum(struct Heap* heap) {
   if (heap->size <= 0) {
      printf("Heap underflow\n");
      return -1;
   }
   return heap->array[0];
}

void maxHeapify(struct Heap* heap, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;

   if (left < heap->size && heap->array[left] > heap->array[largest])
      largest = left;

   if (right < heap->size && heap->array[right] > heap->array[largest])
      largest = right;

   if (largest != i) {
      swap(&heap->array[i], &heap->array[largest]);
      maxHeapify(heap, largest);
   }
}

int heapExtractMax(struct Heap* heap) {
   int max;
   if (heap->size <= 0) {
      printf("Heap underflow\n");
      return -1;
   }

   max = heap->array[0];
   heap->array[0] = heap->array[heap->size - 1];
   heap->size--;
   maxHeapify(heap, 0);

   return max;
}

void heapIncreaseKey(struct Heap* heap, int index, int key) {
   if (key < heap->array[index])
      return;

   heap->array[index] = key;

   while (index > 0 && heap->array[(index - 1) / 2] < heap->array[index]) {
      int parent = (index - 1) / 2;
      swap(&heap->array[index], &heap->array[parent]);
      index = parent;
   }
}

void insert(struct Heap* heap, int value) {
```

```
    if (heap->size == heap->capacity) {
        printf("Heap overflow\n");
        return;
    }

    heap->size++;
    heap->array[heap->size - 1] = NEG_INF;
    heapIncreaseKey(heap, heap->size - 1, value);
}

void printHeap(struct Heap* heap) {
    for (int i = 0; i < heap->size; i++)
        printf("%d ", heap->array[i]);
    printf("\n");
}

int main() {
    struct Heap* heap = createHeap(100);

    insert(heap, 40);
    insert(heap, 20);
    insert(heap, 50);
    insert(heap, 10);
    insert(heap, 30);

    printf("Heap: ");
    printHeap(heap);

    printf("Max: %d\n", heapMaximum(heap));

    printf("Extract: %d\n", heapExtractMax(heap));
    printHeap(heap);

    printf("Extract: %d\n", heapExtractMax(heap));
    printHeap(heap);

    return 0;
}
```

Rubric 1

Criteria	Marks
Correct program structure and function usage	1
Correct heap insertion and priority queue creation	1
Correct implementation of `heapMaximum()`, `heapExtractMax()`, and `maxHeapInsert()`, `heapIncreaseKey()`	4
Proper maintenance of max-heap property after each operation	3
Correct output	1
Total	10

Solution 2

```
    #include <stdio.h>
#include <stdlib.h>

typedef struct Node{
    int key;
    struct Node *left,*right,*p;
} Node;

typedef struct Tree{
    Node *root;
} Tree;

Node* newNode(int key){
    Node* n=(Node*)malloc(sizeof(Node));
    n->key=key; n->left=NULL; n->right=NULL; n->p=NULL;
    return n;
}

void insertNode(Tree* T,int key){
    Node* z=newNode(key);
    Node* x=T->root;
    Node* y=NULL;

    while(x!=NULL){
        y=x;
        if(z->key<x->key) x=x->left;
```

```
      else x=x->right;
   }

   z->p=y;

   if(y==NULL) T->root=z;
   else if(z->key<y->key) y->left=z;
   else y->right=z;
}

Node* treeSearch(Node* x,int k){
   if(x==NULL || k==x->key) return x;
   if(k<x->key) return treeSearch(x->left,k);
   else return treeSearch(x->right,k);
}

Node* treeMin(Node* x){
   while(x->left!=NULL) x=x->left;
   return x;
}

void transplant(Tree* T,Node* u,Node* v){
   if(u->p==NULL)
      T->root=v;
   else if(u==u->p->left)
      u->p->left=v;
   else
      u->p->right=v;

   if(v!=NULL)
      v->p=u->p;
}

void treeDelete(Tree* T,Node* z){
   Node* y;

   if(z->left==NULL){
      transplant(T,z,z->right);
   }
   else if(z->right==NULL){
      transplant(T,z,z->left);
   }
   else{
      y=treeMin(z->right);

      if(y->p!=z){
         transplant(T,y,y->right);
         y->right=z->right;
         y->right->p=y;
      }

      transplant(T,z,y);
      y->left=z->left;
      y->left->p=y;
   }

   free(z);
}

void inorder(Node* x){
   if(x==NULL) return;
   inorder(x->left);
   printf("%d -> ",x->key);
   inorder(x->right);
}

int main(){
   int i;
   int arr[]={50,30,70,20,40,60,80};
   int deleteKeys[]={20,30,50};

   Tree T;
   T.root=NULL;

   for(i=0;i<7;i++) insertNode(&T,arr[i]);

   printf("Initial inorder traversal: ");
   inorder(T.root);
   printf("NULL\n");

```

```
    for(i=0;i<3;i++){
       Node* z=treeSearch(T.root,deleteKeys[i]);
       if(z==NULL){
          printf("\nKey %d not found in the tree.\n",deleteKeys[i]);
       } else {
          printf("\nDeleting %d:\n",deleteKeys[i]);
          treeDelete(&T,z);
          printf("Inorder traversal after deletion: ");
          inorder(T.root);
          printf("NULL\n");
       }
    }
    return 0;
}
```

Rubric

Criteria	Marks
Correct program structure and function usage	2
Correct BST insertion and tree creation	2
Correct implementation of `transplant()` and deletion cases (leaf, one child, two children)	3
Proper maintenance of BST properties after deletion	2
Correct output	1
Total	10